WEEK 2 · LESSON 7 OF 24 · HANDOUT

Your First LLM API Call

Study time: **about 50 minutes** (reading, video and activity)

▶ **Watch:** https://www.youtube.com/watch?v=EWvNQjAaOHw — open the lesson online to watch inside the course.

🎓 **Go deeper:** Andrej Karpathy: Intro to Large Language Models (1-hour talk) — https://www.youtube.com/watch?v=zjkBMFhNj_g

🎓 **Go deeper:** CS50 Tech Talk: Large Language Models and the End of Programming — https://www.youtube.com/watch?v=JhCl-GeT4jw

Your goal in this lesson: Call a large language model from Python, keep your API key safe, read the parts of a response (text, stop reason, token usage) and estimate what a call costs.

Chat app vs API

A chat app (claude.ai, ChatGPT, Gemini) is a website for people. An **API** (application programming interface) lets **your code** send a request and get the answer back — so you can build the model into your own website, WhatsApp bot, school system or business tool.

```
your program  --(prompt, settings, API key)-->  LLM provider
your program  <--(answer, token usage)--------  LLM provider
```

Step 1 — Get an API key, and keep it secret

1. Create an account in the provider's console (for Claude: console.anthropic.com) and create an **API key**.
2. API use is paid per token. A few dollars of credit is enough for this whole course with a small, fast model. Set a **spending limit** in the console.
3. Free options exist too: some providers offer free tiers (see the Gemini example below), and Lesson 22 shows how to run open models for free in Colab.

An API key is like a password **and** a credit card. Never paste it into code you share, a GitHub repository, a screenshot or a chat. In Colab, open the **Secrets** panel (key icon on the left), add a secret named `ANTHROPIC_API_KEY`, and let your notebook read it.

Step 2 — Set up

```
import importlib.util, os, subprocess, sys

if importlib.util.find_spec("anthropic") is None:          # install once per Colab session
    subprocess.run([sys.executable, "-m", "pip", "install", "-q", "anthropic"], check=True)

try:
    from google.colab import userdata                     # Colab: key saved in the Secrets
panel (key icon)
    os.environ["ANTHROPIC_API_KEY"] = userdata.get("ANTHROPIC_API_KEY")
except ImportError:
    pass                                                    # not in Colab: set
ANTHROPIC_API_KEY in your environment
except Exception:
    print("Add a secret called ANTHROPIC_API_KEY in Colab's Secrets panel, then run this cell
again.")

import anthropic

client = anthropic.Anthropic()        # reads ANTHROPIC_API_KEY from the environment
MODEL = "claude-haiku-5-5"            # fast and low-cost; check docs.claude.com for current
model names

def ask(prompt, system=None, max_tokens=500):
    extra = {"system": system} if system else {}
    response = client.messages.create(model=MODEL, max_tokens=max_tokens,
                                      messages=[{"role": "user", "content": prompt}], **extra)
    return response.content[0].text
```

`ask()` is a small helper we reuse all week: one prompt in, one text answer out.

Step 3 — Your first call

```
response = client.messages.create(
    model=MODEL,
    max_tokens=300,
    system="You are a friendly tutor for students in Rwanda. Use simple English.",
    messages=[{"role": "user", "content": "In two sentences, what is an API?"}],
)
print(response.content[0].text)
```

The answer is **different every time you run it** (Lesson 3: sampling). Your result will not match anyone else's word for word.

Step 4 — Read the whole response

```
print("stop reason: ", response.stop_reason)
print("input tokens:", response.usage.input_tokens)
print("output tokens:", response.usage.output_tokens)
print("model:       ", response.model)
```

Field	Meaning
`content`	A list of blocks; for plain answers, `content[0].text` is the text
`stop_reason`	`end_turn` (finished), `max_tokens` (cut off — raise the limit), `tool_use` (wants a tool, Lesson 9)
`usage`	Tokens read and written — this is what you pay for

`max_tokens` is a **ceiling** on the answer length, not a target. If `stop_reason` is `max_tokens`, the answer was cut off mid-sentence.

Step 5 — Estimate the cost

Prices are listed per **million** tokens and change over time — always check the provider's current price list. With example prices:

```
PRICE_IN = 1.00     # example US$ per million input tokens  (check the real price list)
PRICE_OUT = 5.00    # example US$ per million output tokens

def cost_usd(usage):
    return usage.input_tokens / 1e6 * PRICE_IN + usage.output_tokens / 1e6 * PRICE_OUT

c = cost_usd(response.usage)
print(f"this call: ${c:.6f}   ->  10,000 similar calls: ${c * 10_000:.2f}")
```

One small call costs a tiny fraction of a cent; costs matter when an app makes thousands of calls a day, or sends long documents every time.

The same idea with another provider

Many providers, including Google's Gemini, also accept the "OpenAI-compatible" format. Google AI Studio offers a free tier for experimenting:

```
if importlib.util.find_spec("openai") is None:
    subprocess.run([sys.executable, "-m", "pip", "install", "-q", "openai"], check=True)
from openai import OpenAI

gemini = OpenAI(api_key=os.environ.get("GEMINI_API_KEY", "paste-in-colab-secrets"),
                base_url="https://generativelanguage.googleapis.com/v1beta/openai/")
GEMINI_MODEL = "gemini-2.5-flash"          # pick a current model name in aistudio.google.com
if os.environ.get("GEMINI_API_KEY"):
    reply = gemini.chat.completions.create(
        model=GEMINI_MODEL,
        messages=[{"role": "system", "content": "Answer in one sentence."},
                  {"role": "user", "content": "What is an API?"}])
    print(reply.choices[0].message.content)
```

The shapes differ slightly — `choices[0].message.content` instead of `content[0].text` — but the ideas (messages, roles, tokens) are the same everywhere. The rest of the course uses the Claude API; adapting to another provider takes a few lines.

Key terms

- **API** — a way for programs to use a service.
- **API key** — your secret credential; whoever has it can spend your money.
- **Messages** — the conversation sent to the model, each with a role.

- **max_tokens** — the most tokens the answer may use.
- **stop_reason** — why the model stopped.
- **Usage** — tokens in and out, the basis of cost.

Remember this: An LLM API call is a list of messages in, text and token usage out — keep the key secret, check why the model stopped, and keep an eye on tokens because they are your cost.

Name: _____ Date: _____

Activity

Task 1

Ask the same question three times. Compare the answers and their token counts.

Task 2

Set `max_tokens=20` and ask for a long explanation. What is the `stop_reason`?

Task 3

Change the system prompt to "Answer only in Kinyarwanda" and ask a simple question. How good is the answer?

Task 4

Calculate the cost of sending a 10-page document (about 5,000 tokens) and getting a 300-token summary, with the example prices.

Self-check

Think about your answers here, then take the quiz online — it is marked instantly and explains every answer.

1. What does an LLM API let you do?
   - A. Train a model for free
   - B. Avoid paying for tokens
   - C. Chat only on a website
   - D. Use the model from your own code and apps
2. Where should you keep your API key in Colab?
   - A. In a public GitHub repo
   - B. In a screenshot
   - C. In the Secrets panel, read by the notebook
   - D. Directly in the code
3. stop_reason is 'max_tokens'. What happened?
   - A. A tool was called
   - B. The answer finished normally
   - C. The answer was cut off at the token limit
   - D. The key is wrong
4. What do you pay for with an LLM API?
   - A. The number of users
   - B. Nothing
   - C. Input and output tokens
   - D. Each second of use
5. Why do two runs of the same prompt give different answers?
   - A. The model samples its tokens
   - B. The API is broken
   - C. Tokens are deleted
   - D. The key changes
6. Which field holds the text of a Claude API answer?
   - A. response.choices
   - B. response.content[0].text
   - C. response.text
   - D. response.answer

© WAMUGII TECH SOLUTIONS · wamugii.kento.rw · Original course material — for enrolled students.